Réseaux de Neurones Artificiels

Rose Bandolo

1er mai 2025

Table de matiere

1. Origine des Réseaux de Neurones Artificiels
2. Évolution des réseaux de neurones artificiels
3. Comprendre le fonctionnement du Perceptron simple
4. Calcul des gradients de la fonction de coût
5. Réseau de Neurone Artificiel simple
6. Fonctions d'activation
7. TP :Implémention d'un perceptron simple avec Numpy

Origine des Réseaux de Neurones Artificiels

- **Années 1940-1950 :** Inspirations biologiques
  - McCulloch et Pitts proposent un modèle formel de neurone binaire.
  - Le modèle est inspiré par le fonctionnement des neurones biologiques.

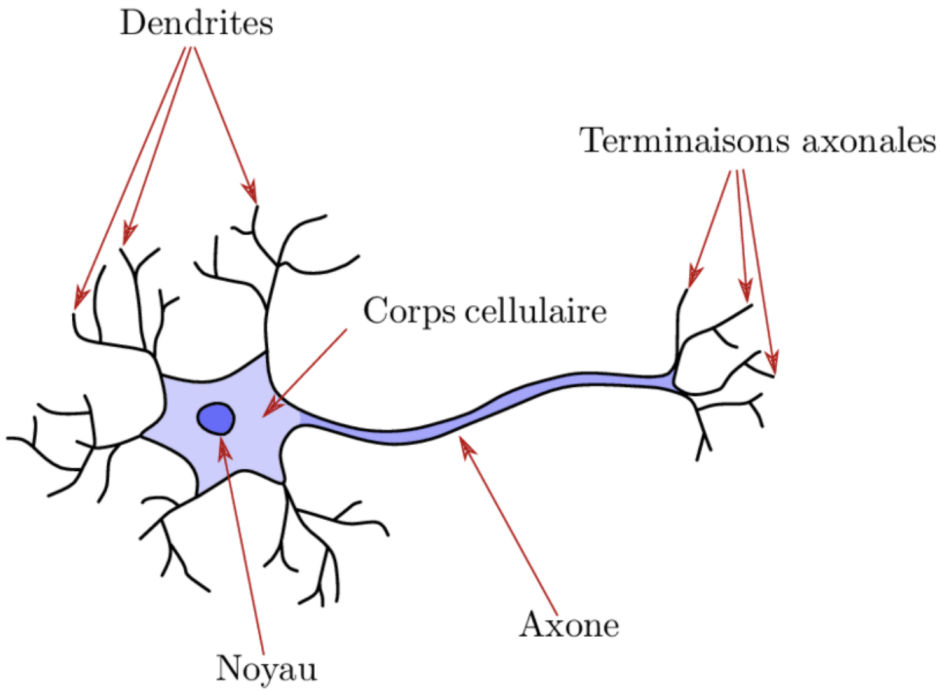
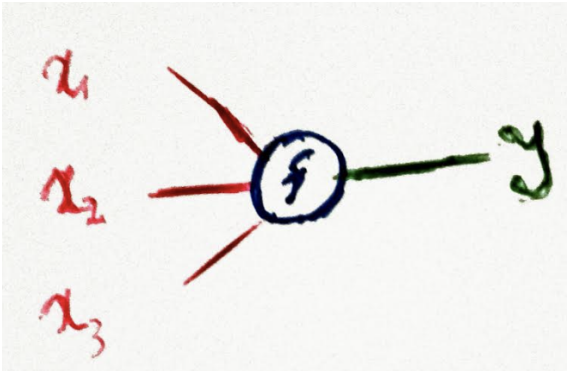


Figure 1 – Réseau de neurone biologique

Les neurones sont des cellules excitables connectées les unes aux autres et ayant pour rôle de transmettre des informations dans notre système

Origine des Réseaux de Neurones Artificiels

McCulloch et Pitts ont modélisé ce fonctionnement en considérant le neurone comme une fonction de transfert f qui prend un certains nombres de signaux en entrée (x_1, x_2, x_3) et renvoi une sortie binaire y



- Agrégation : $f = w_1 x_1 + w_2 x_2 + w_3 x_3$
- Activation : $\begin{cases} y = 1 & \text{si } f \geq 0 \\ y = 0 & \text{sinon.} \end{cases}$

Origine des Réseaux de Neurones Artificiels

- **Années 1950-1960 :** Premier modèle d'apprentissage : Le Perceptron
  - Frank Rosenblatt développe les perceptrons, des modèles de neurones à seuil.

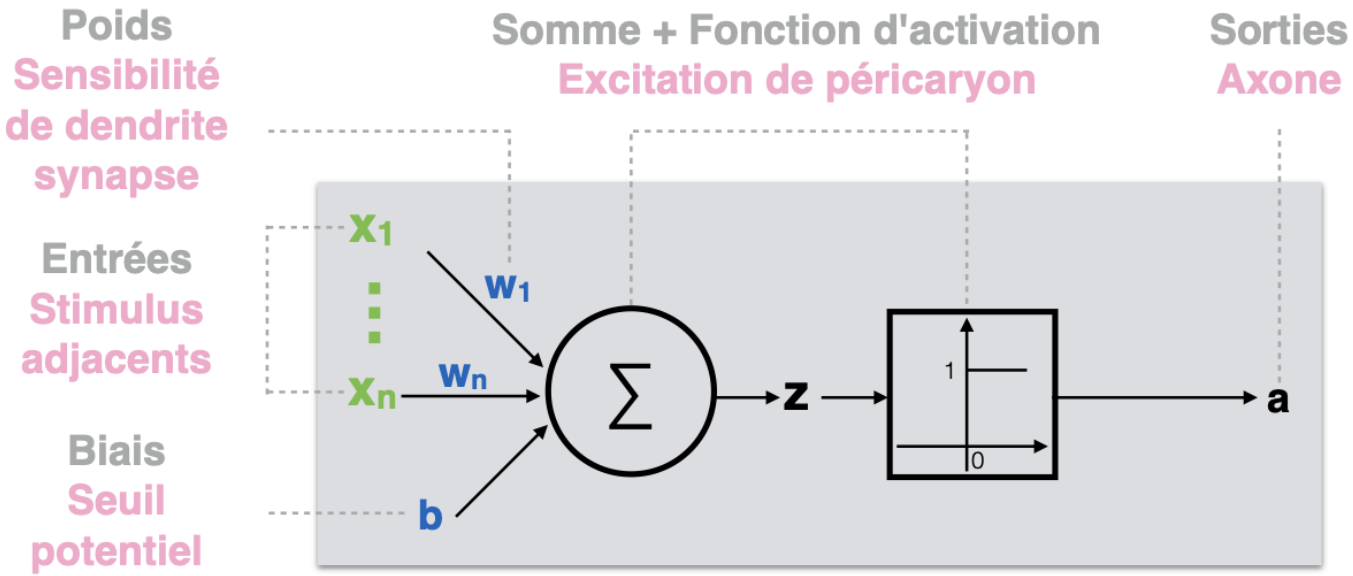
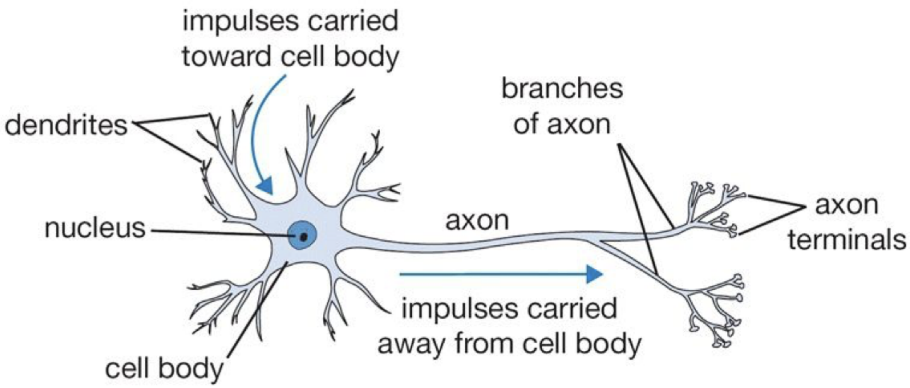


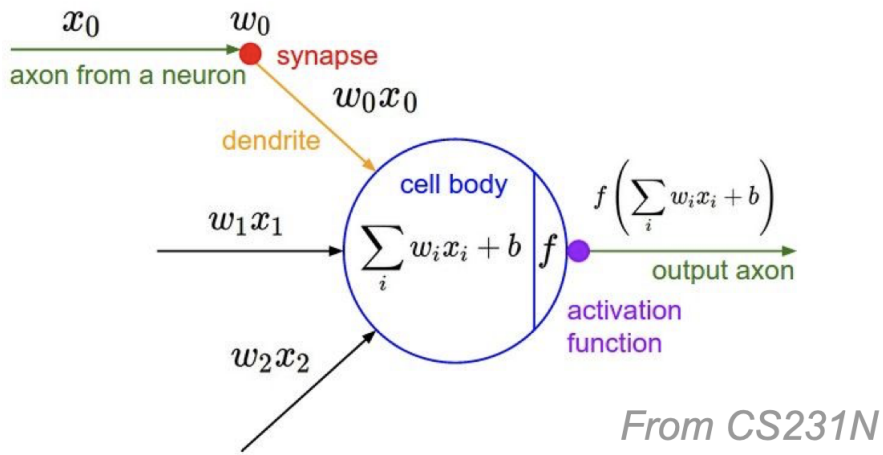
Figure 2 – Structure d'un perceptron

Origine des Réseaux de Neurones Artificiels

- **Le perceptron** prend en compte le niveau de signal provenant de différents neurones.
- Il est créé pour imiter mathématiquement les principes de fonctionnement du neurone.
- Il possède plusieurs entrées binaires x1,x2,...,xn (dendrites)
- à chaque entrée correspond un poids w1,w2,...,wn (sensibilité des synapses).
- La somme de chaque entrée multipliée par son poids est ensuite comparée à un seuil b (potentiel de seuil). La sortie est égale à 1 si la somme est supérieure au seuil (neurone actif). Sinon, la sortie est égale à 0 (neurone inhibé)

Origine des Réseaux de Neurones Artificiels

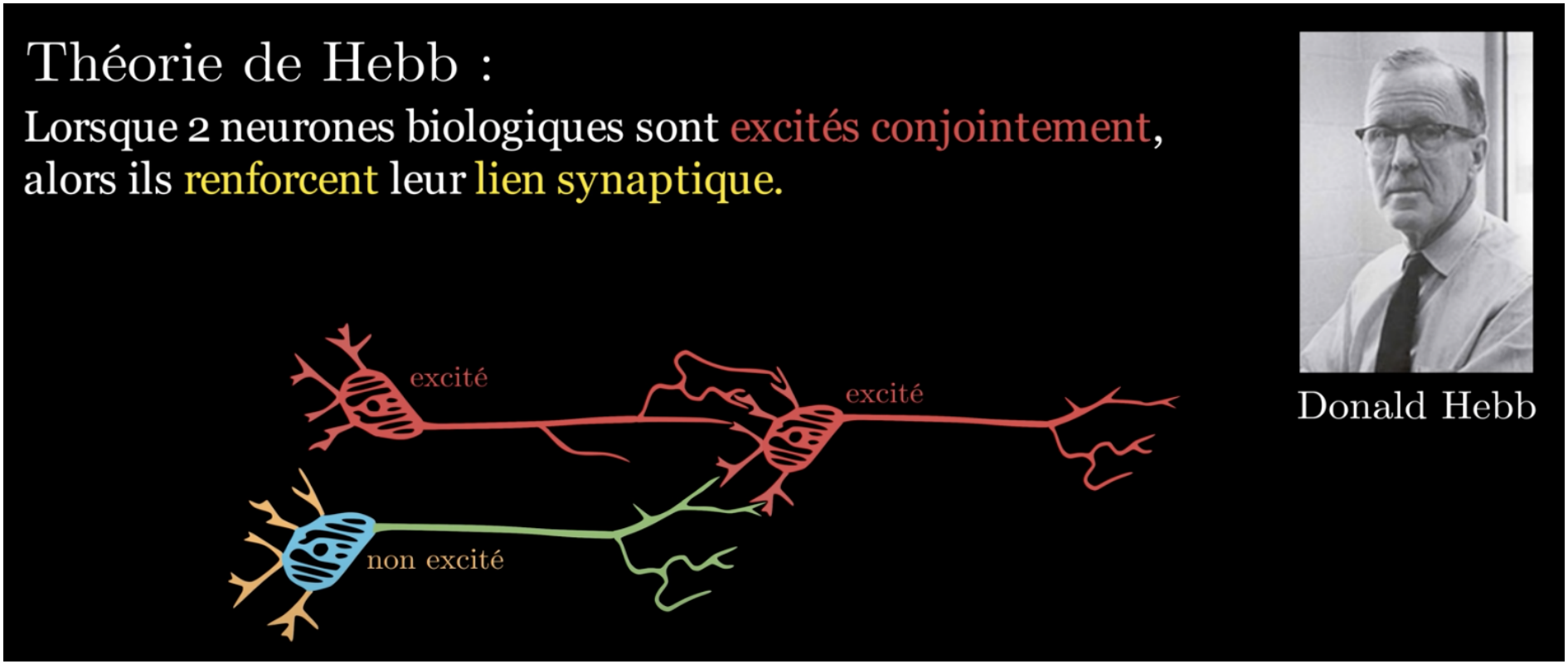




From CS231N

Origine des Réseaux de Neurones Artificiels

- L'algorithme d'apprentissage de Frank Rosenblatt repose sur **la théorie de Hebb**



Origine des Réseaux de Neurones Artificiels

- Entraîner un neurone artificiel sur des données de références (X,y) pour que celui ci renforce ses paramètres W a chaque fois q'une entrée X est activée en même temps que la sortie y dans les données.

$$W = W + \alpha(y_{true} - y)X \tag{1}$$

y_{true} : est la sortie de reference
y : la sortie predite par le neurone
α : la vitesse d'apprentissage
X les données en entrée

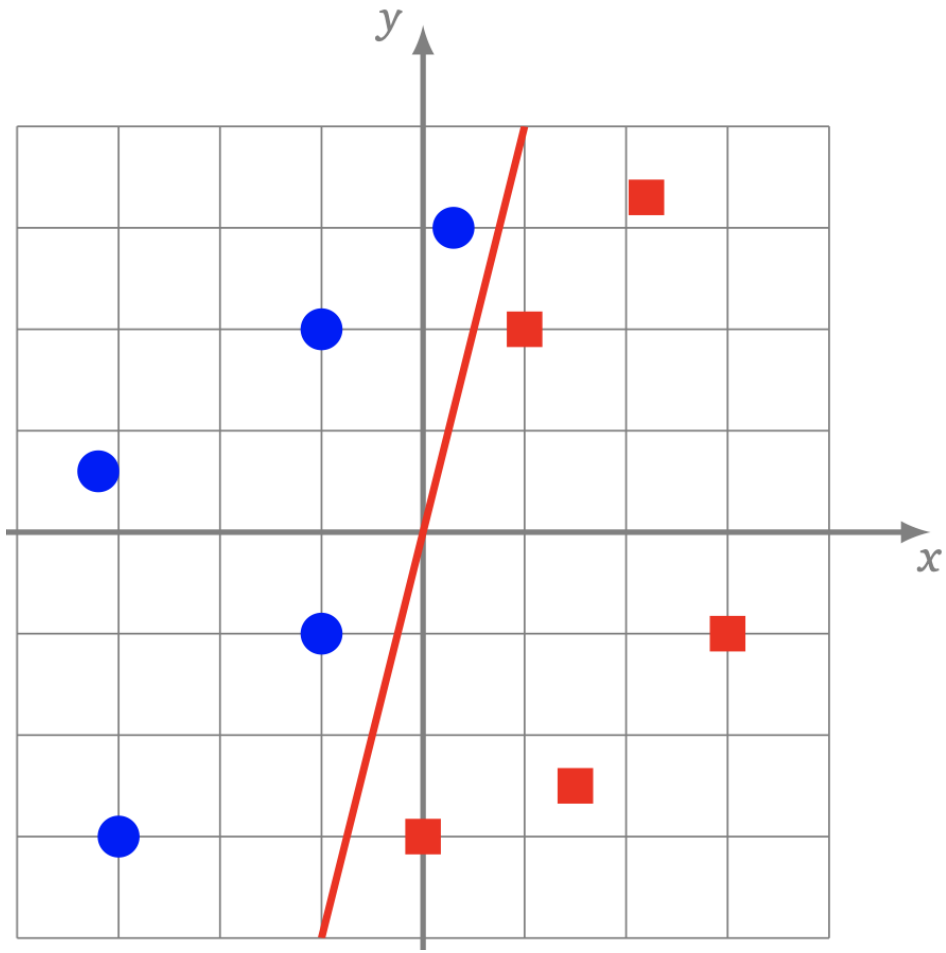
Évolution des Réseaux de Neurones Artificiels

- **Années 1980-1990 :** Ère des Réseaux de Neurones
  - Découverte de l'algorithme de rétropropagation pour l'entraînement de réseaux multicouches.
  - Les réseaux de neurones multicouches (MLP) deviennent populaires.
  - Cependant, les limites en termes de calcul et de données ralentissent la recherche.
- **Années 2000-2010 :** Apprentissage Profond Moderne
  - Avancements en calcul, grandes quantités de données et architectures plus complexes.
  - Émergence de réseaux de neurones profonds.
  - Succès dans diverses applications, notamment la vision par ordinateur et le traitement du langage naturel.
- **Années 2010-2020 :** Résurgence de l'IA
  - Explosion de la recherche et de l'industrie autour de l'apprentissage profond.
  - Réseaux de neurones convolutifs (CNN), réseaux de neurones récurrents (RNN) et plus encore.

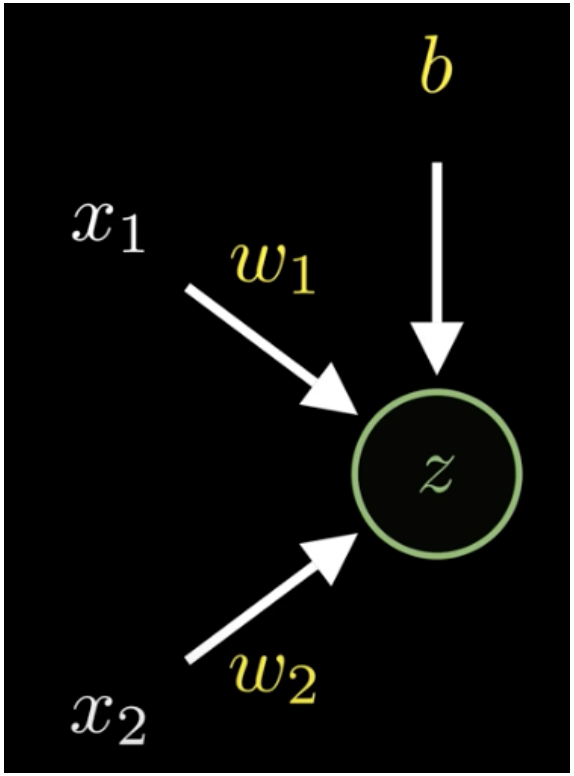
Comprendre le fonctionnement du Perceptron simple

Exemple : Trouver un perceptron qui distingue les carrés des ronds.

y	x_1	x_2
1	2	1.3
1	1	2.3
1	0	4
0	1	2
0	2	0.3

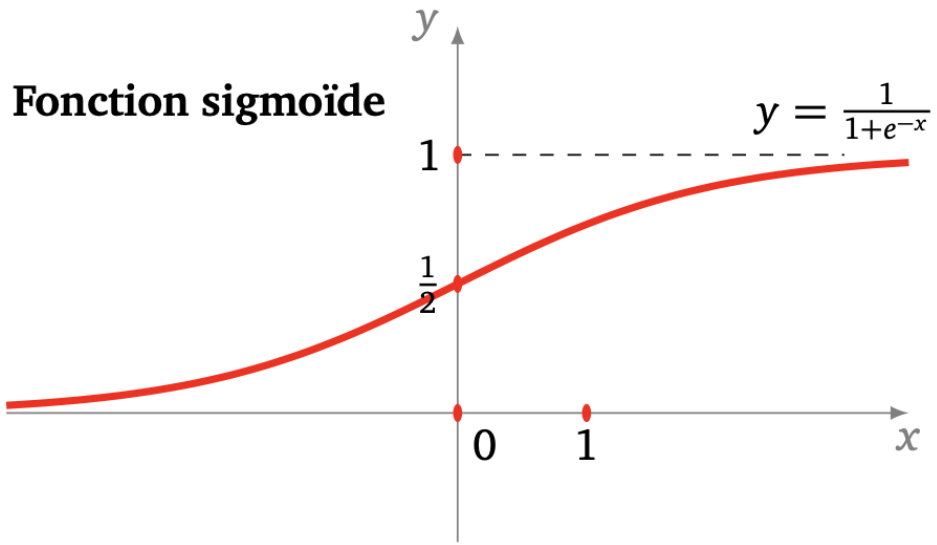


Comprendre le fonctionnement du Perceptron simple



- Fonction d'agrégation : $z(x_1, x_2) = w_1 x_1 + w_2 x_2 + b$
- Activation : $\begin{cases} y = 1 & \text{si } z \geq 0 \\ y = 0 & \text{sinon.} \end{cases}$

La fonction sigmoïde ou fonction logistique



- Pour notre question « Est-ce que c'est un rond ou un carré ? »,
- si la sortie vaut 0.8, cela signifie « c'est bien un rond avec 80% de certitude ».
- Si la sortie vaut 0.1 alors ce n'est probablement pas un ron.

Comprendre le fonctionnement du Perceptron simple

La fonction de coût

- Permet de quantifier les erreurs du modèle
- calculer la distance entre les sorties z et les vraies données y
- **la fonction coût log loss** :

$$L = -\frac{1}{m}\sum_{i=1}^{m} y_i log(a_i) + (1 - y_i) log(1 - a_i) \tag{2}$$

m : nombres de données
y_i : donnée i
a_i : sortie i

La fonction de coût log loss

- tire son origine de la formule de la vraisemblance :

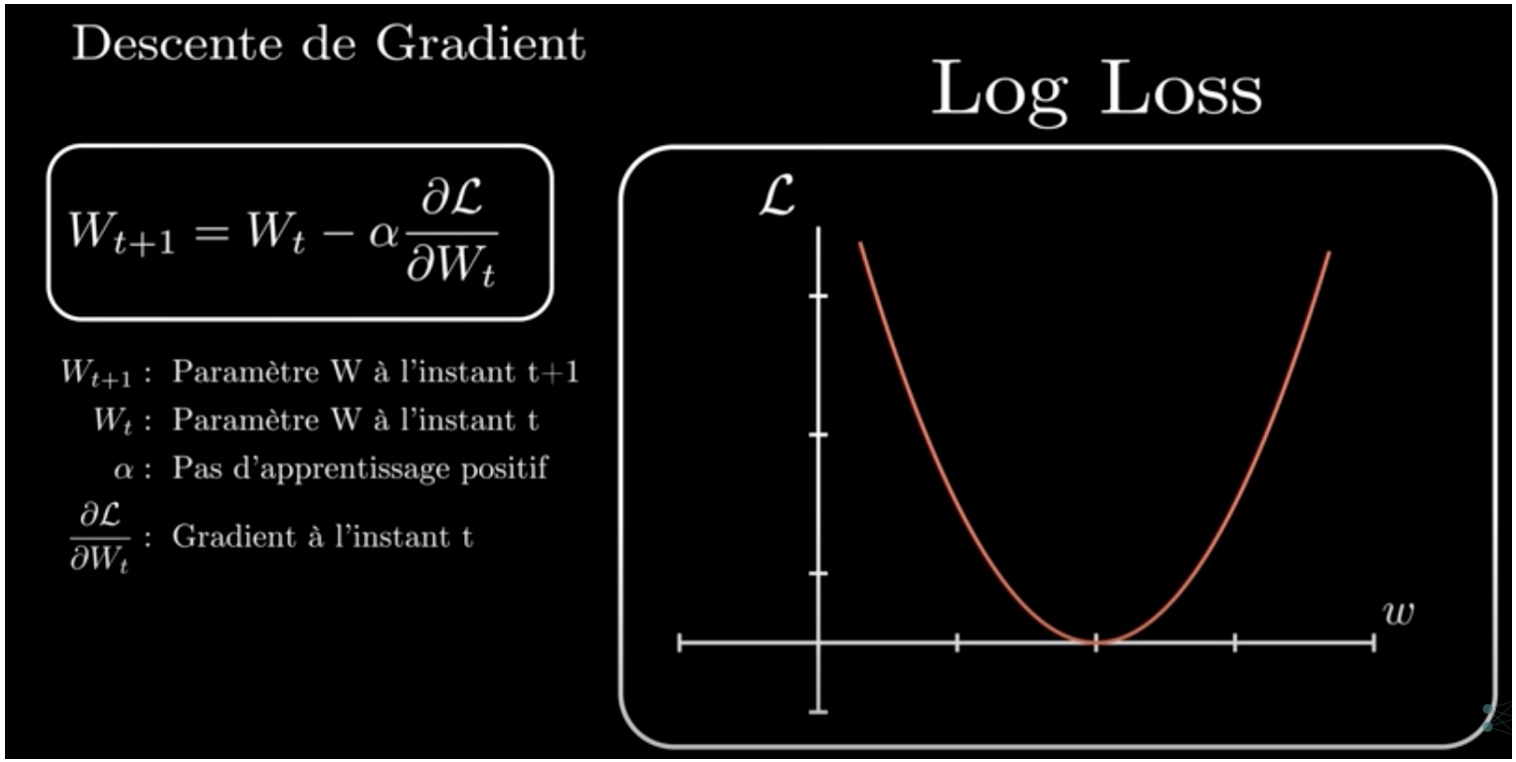
$$L = \prod_{i=1}^{n} (a_i)^{y_i} \cdot (1 - a_i)^{1-y_i}$$

- plus on a de nombre plus cette fonction tend vers 0.
- pour eviter qu'elle converge on utilise le logarithme
- on veut avoir un modele qui maximise la vraisemblance
- maximiser $f(x)$ = minimiser $-f(x)$

Comprendre le fonctionnement du Perceptron simple

La descente du gradient

- Ajuster les paramètres W et b qui minimisent les erreurs du modèle ie qui minimisent notre fonction de coût
- calculer le gradient ou encore la dérivée de la fonction de coût



Comprendre le fonctionnement du Perceptron simple

Les formules des différents gradients sont données par :

$$\frac{\partial L}{\partial w_1} = \frac{1}{m}\sum_{i=1}^{m}(a_i - y_i)x_1$$

$$\frac{\partial L}{\partial w_2} = \frac{1}{m}\sum_{i=1}^{m}(a_i - y_i)x_2$$

$$\frac{\partial L}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(a_i - y_i)$$

Calcul des Gradients de la Fonction de Coût

Nous voulons calculer les gradients de la fonction de coût :

$$L = -\frac{1}{m}\sum_{i=1}^{m} y_i \log(a_i) + (1 - y_i)\log(1 - a_i)$$

Cela revient à calculer $\frac{\partial L}{\partial w_1}$, $\frac{\partial L}{\partial w_2}$ et $\frac{\partial L}{\partial b}$. Pour cela on fait appel à **la règle de chaîne** .

1. Calculons la dérivée partielle par rapport à w_1 : $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial a} * \frac{\partial a}{\partial z} * \frac{\partial z}{\partial w_1}$

$$\frac{\partial L}{\partial a} = -\frac{1}{m}\sum \frac{y}{a} - \frac{1-y}{1-a}$$

Nous pouvons encore ecrire a comme $a = g \circ f = g(f(z)$

$$\begin{cases} g = \frac{1}{f} & g^{'} = \frac{-1}{f^2} \\ f = 1 + e^{-z} & f^{'} = -e^{-z} \end{cases} \quad (g \circ f)^{'} = g^{'}(f(z)) * f^{'}(z)$$

Calcul des Gradients de la Fonction de Coût

$$
\begin{aligned}
\frac{\partial a}{\partial z} &= g^{'}(f(z)) * f^{'}(z) \\
&= -\frac{1}{(1+e^{-z})^2} * (-e^{-z}) \\
&= \frac{e^{-z}}{(1+e^{-z})^2} \\
&= \frac{1}{1+e^{-z}} * \frac{e^{-z}}{1+e^{-z}} \\
&= \frac{1}{1+e^{-z}} * \frac{e^{-z}+1-1}{1+e^{-z}} \\
&= \frac{1}{1+e^{-z}} * (\frac{e^{-z}+1}{1+e^{-z}} - \frac{1}{1+e^{-z}}) \\
&= a(1-a)
\end{aligned}
$$

$$\frac{\partial a}{\partial z} = a(1-a)$$

Calcul des Gradients de la Fonction de Coût

calculons la derivee de z par a w_1

$$z = w_1 x_1 + w_2 x_2 + b$$

$$\frac{\partial z}{\partial w_1} = x_1$$

$$\begin{aligned} \frac{\partial l}{\partial w_1} &= -\frac{1}{m} \sum \left(\frac{y}{a} - \frac{1-y}{1-a}\right) * a(1-a) * x_1 \\ &= -\frac{1}{m} \sum (y-a) x_1 \end{aligned}$$

Calcul des Gradients de la Fonction de Coût

$$\begin{aligned}
\frac{\partial l}{\partial w_1} &= -\frac{1}{m}\sum\left(\frac{y}{a} - \frac{1-y}{1-a}\right) * a(1-a) * x_1 \\
&= -\frac{1}{m}\sum(y(1-a) - (1-y)a)x_1 \\
&= -\frac{1}{m}\sum(y - ya - a + ya)x_1 \\
&= -\frac{1}{m}\sum(y-a)x_1
\end{aligned}$$

Réseau de Neurone Artificiel simple

- Type de réseau neuronal avec un petit nombre de couches cachées.
- Ne comportent qu'une ou deux couches cachées.
- La figure ci-dessous montre un réseau neuronal avec 1 couche cachée, 1 couche d'entrée et 1 couche de sortie.

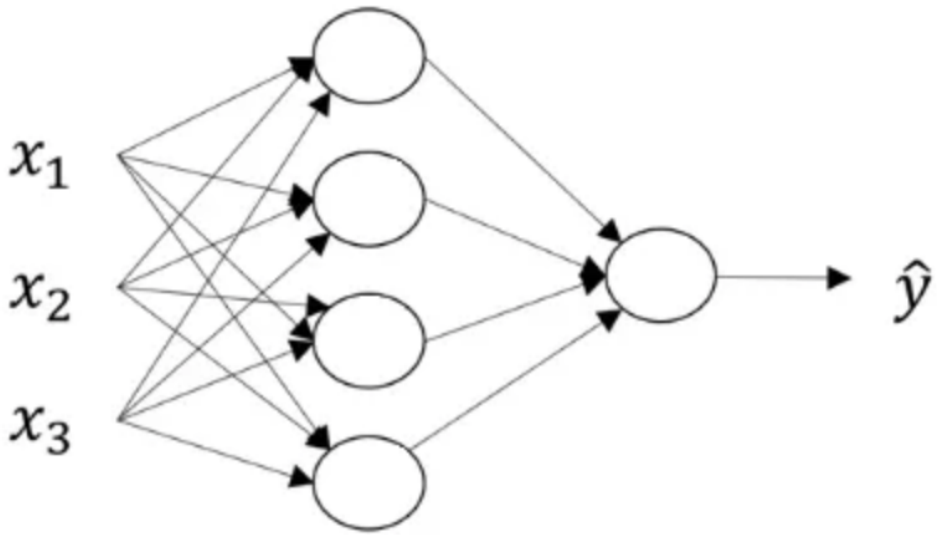


Figure 4 – Réseau de neurone artificiel simple

Réseau de Neurone Artificiel simple : Fonctionnement

- Le **Feed Forward** est la première phase dans l'algorithmique des réseaux de neurones.
- Il s'agit de la propagation des données d'entrée à travers le réseau pour obtenir une prédiction.
- Chaque neurone dans une couche transmet ses sorties pondérées aux neurones de la couche suivante.
- La somme pondérée est passée à travers une fonction d'activation pour produire la sortie du neurone.
- Ce processus se répète couche par couche jusqu'à ce que la prédiction finale soit obtenue.

Réseau de Neurone Artificiel simple : Fonctionnement

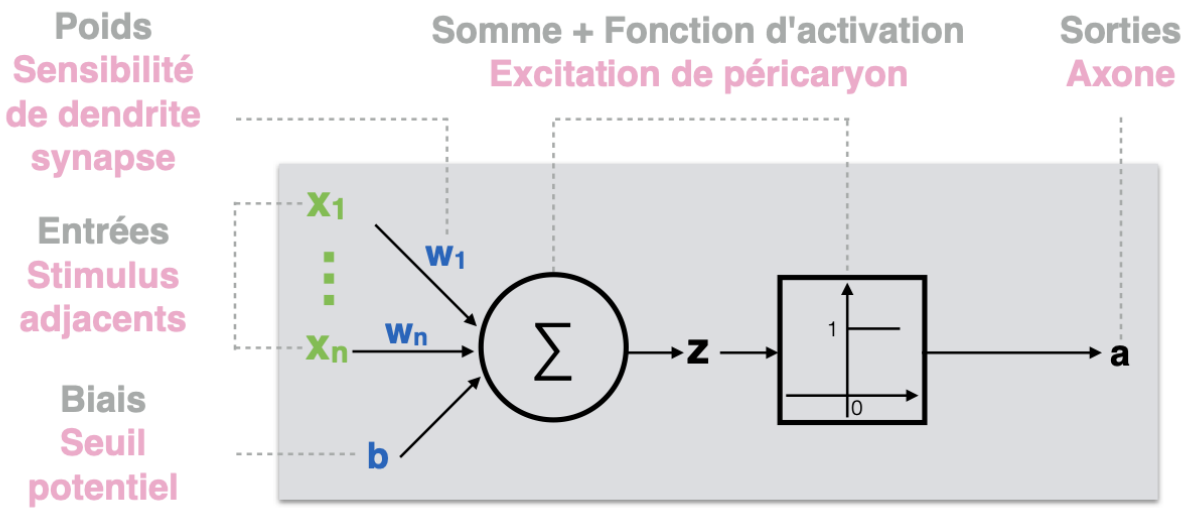


Figure 5 – Structure du perceptron

- Calcul de la somme pondérée : $z = \sum_{i=1}^{n} w_i x_i + b$
- Utilisation de la fonction d'activation : $a = f(z)$

Réseau de Neurone Artificiel simple : Fonctionnement

La couche cachée comprend plusieurs neurones, chacun d'entre eux effectuant les deux calculs ci-dessus. Les 4 neurones présents dans la couche cachée de notre réseau neuronal peu profond effectuent les calculs suivants :

$$z_1^{(1)} = W_1^{(1)T} x + b_1^{(1)}, \quad a_1^{(1)} = \sigma(z_1^{(1)}) \tag{3}$$

$$z_2^{(1)} = W_2^{(1)T} x + b_2^{(1)}, \quad a_2^{(1)} = \sigma(z_2^{(1)}) \tag{4}$$

$$z_3^{(1)} = W_3^{(1)T} x + b_3^{(1)}, \quad a_3^{(1)} = \sigma(z_3^{(1)}) \tag{5}$$

$$z_4^{(1)} = W_4^{(1)T} x + b_4^{(1)}, \quad a_4^{(1)} = \sigma(z_4^{(1)}) \tag{6}$$

Réseau de Neurone Artificiel simple : Fonctionnement

Le chiffre en exposant i indique le numéro de la couche et le chiffre en exposant j indique le numéro du neurone dans une couche particulière.

- X est le vecteur d'entrée composé de 3 caractéristiques.
- $W_j^{(i)}$ est le poids associé au neurone j présent dans la couche i.
- $b_j^{(i)}$ est le biais associé au neurone j présent dans la couche i.
- $z_j^{(i)}$ est la sortie intermédiaire associée au neurone j présent dans la couche i.
- $A_j^{(i)}$ est la sortie finale associée au neurone j présent dans la couche i.
- σ est la fonction d'activation sigmoïde. Mathématiquement, elle est définie comme suit :

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{7}$$

Réseau de Neurone Artificiel simple : Fonctionnement

Comme nous pouvons le constater, les 4 équations ci-dessus semblent redondantes. Par conséquent, nous les vectoriserons comme suit :

$$Z^{(1)} = W^{(1)T} X + b^{(1)}$$
$$A^{(1)} = \sigma(Z^{(1)})$$

- La première équation calcule toutes les sorties intermédiaires Z en une seule multiplication matricielle.
- La deuxième équation calcule toutes les activations A en une seule multiplication matricielle.

Fonctions d'Activation

- **Fonctions d'Activation :**
  - Introduisent la non-linéarité dans le modèle.
  - Permettent au réseau de capturer des relations complexes dans les données.
- **Fonctions d'Activation Courantes :**
  - Sigmoïde :

$$f(z) = \frac{1}{1 + e^{-z}} \tag{8}$$

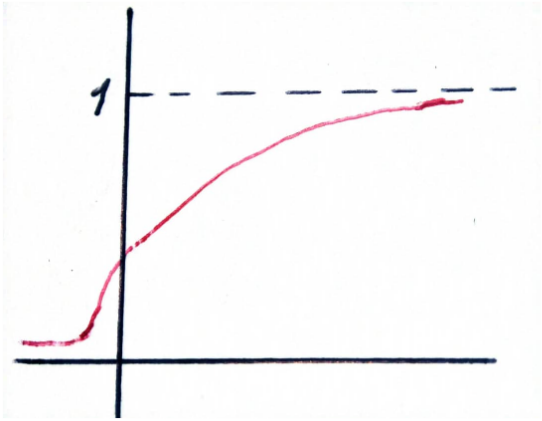
  - Tanh (Tangente hyperbolique) :

$$f(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \tag{9}$$

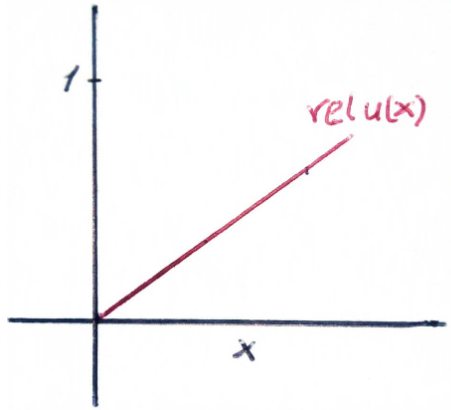
  - ReLU (Rectified Linear Unit) :

$$f(z) = \max(0, z) \tag{10}$$

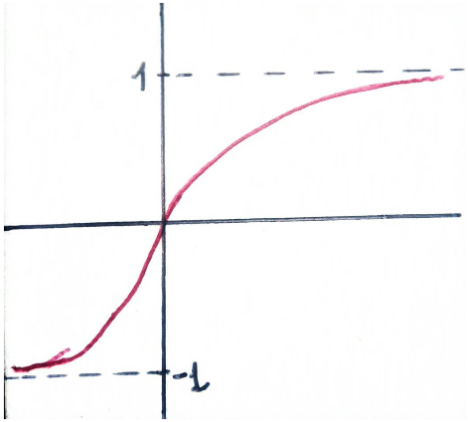
Fonctions d'Activation



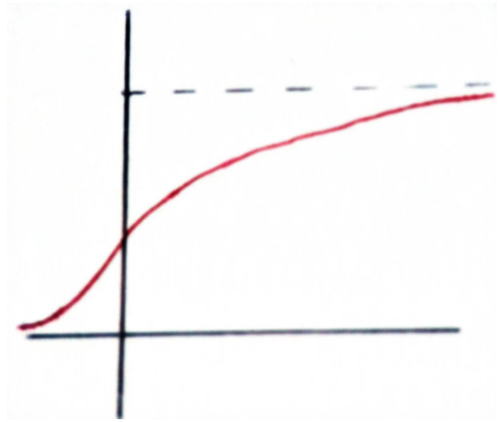
(a) Sigmoid/Logistic



(b) Rectified linear unit



(c) Hyperbolic Tangent



(d) Softmax

Figure 6 – Fonctions d'activation les plus courantes

References

- https://www.youtube.com/watch?v=XUFLq6dKQok&list=PLO_fdPEVlfKoanjvTJbIbd9V5d9Pzp8Rw
- https://www.math.u-bordeaux.fr/~jbigot/Site/Enseignement_files/Intro_DeepLearning.pdf
- https://www.math.univ-toulouse.fr/~besse/Wikistat/pdf/st-m-hdstat-rnn-deep-learning.pdf